

3;

Assignment - 23

Q-11 Features of Java PL

- 1) Simple & Easy to learn
- 2) Pure OOP (everything considered class i.e. objects).
- 3) Compiled & interpreted PL
- 4) Virtual Machine Based PL
- 5) Architecture Neutral (any mp)
- 6) Platform Independent (any OS)
- 7) Virtual Based bcoz - JVM is virtual machine of Java
- 8) Dynamic in Nature (everything decided at runtime)

- 1) Supports inbuilt concept of multi-threading
- 2) Robust PL -
- 3) Supports almost all databases in world (Oracle, MySQL, etc.)
- 4) Automatic Garbage Collection.
- 5) Not in C & C++
- 6) GUI Programming.
- 7) Supports all concepts of Networking.

1) Platform Independent & Architecture Neutral

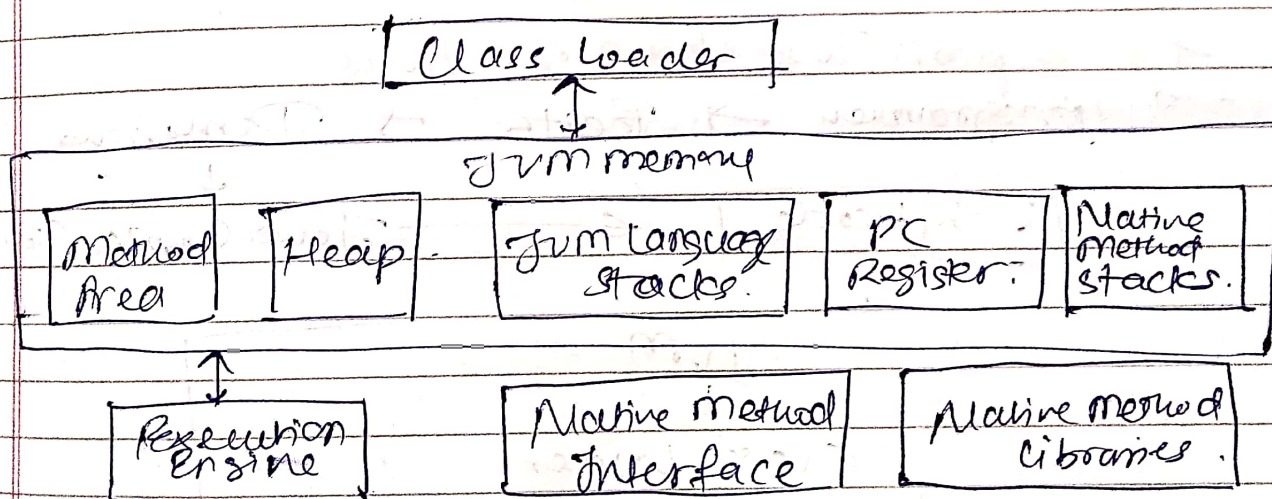
→ Platform Independence - Can be executed on any type of micro processor

Ex: Intel, Motorola, AMD, etc.

2) Architecture Neutral - Executed on any type of OS.

Ex: Windows, Mac, Linux.

3) Explain JVM architecture in detail
JVM Internals.



* JVM - Java Virtual Machine

Heart of JRE

Responsible for verifying bytecode & executing bytecode, line by line [Interpreter].

mini OS.

memory m,
 2) file m.
 3) process m.

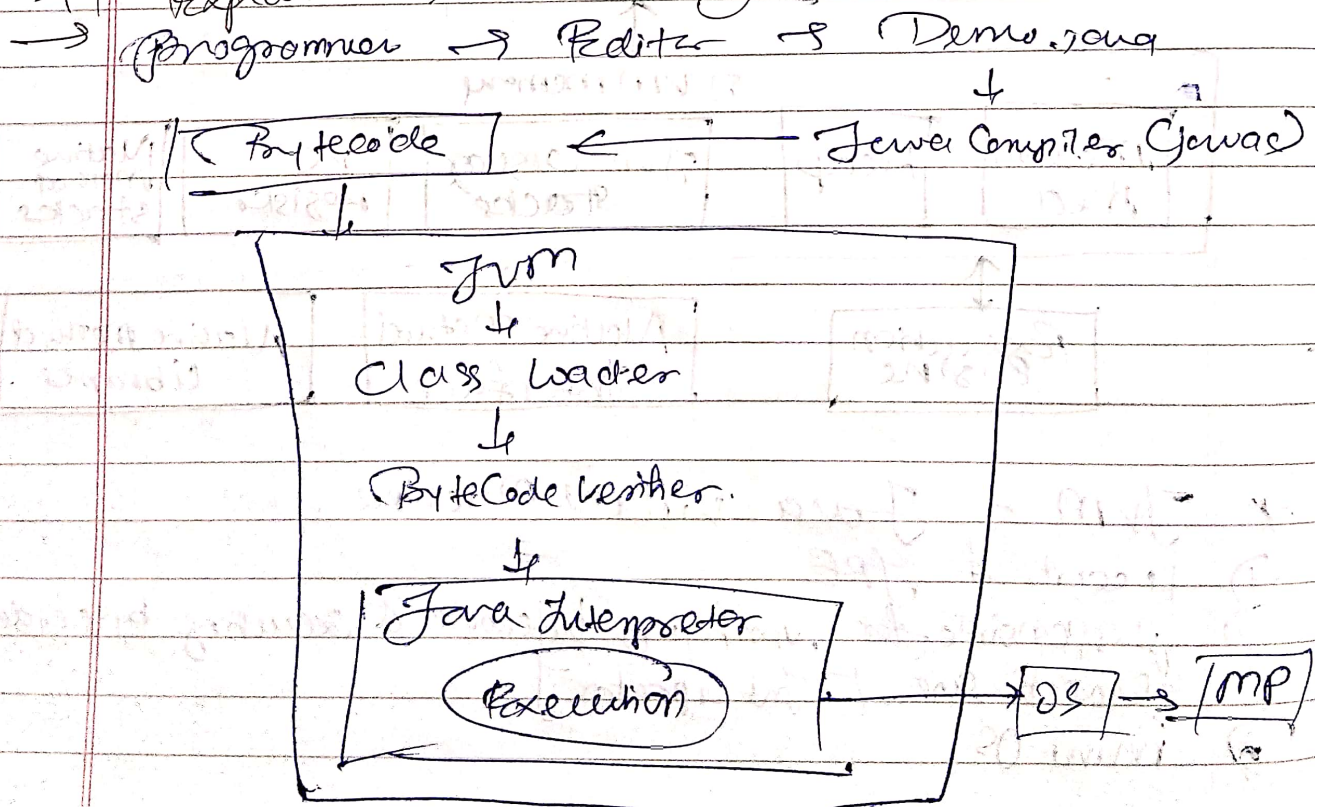
4) Hardware Abstraction
 5) CPU scheduling

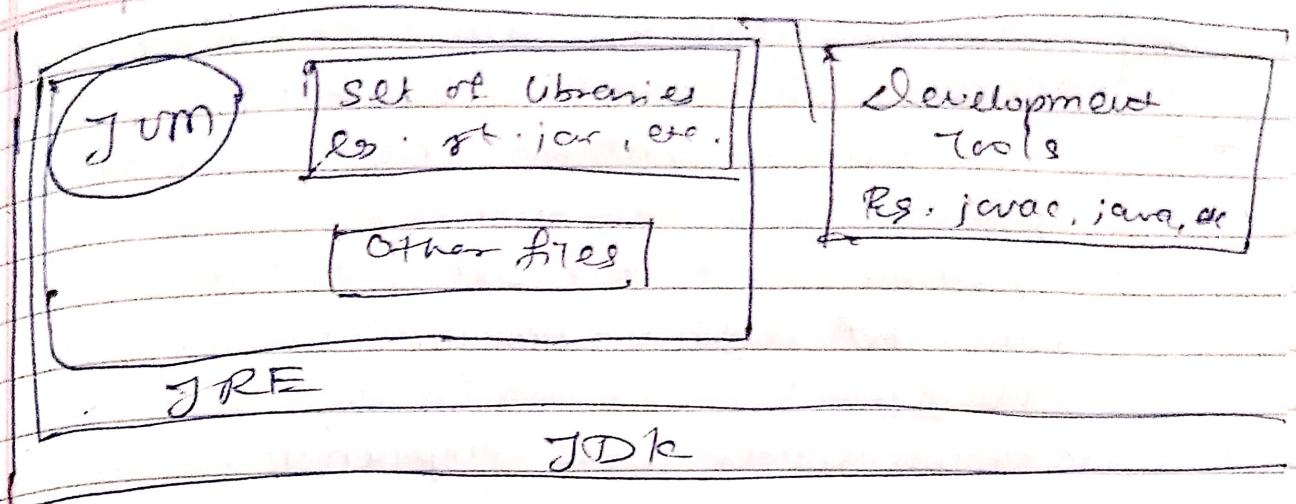
classmate

Date
 Page

- 1) Internally perform 5 tasks like OS.
 When we execute our 1st Java Program, JVM gets loaded in memory.
- 2) After loading JVM will take some part of memory (RAM) from OS.
- 3) Now it's JVM duty to manage all its memory management requirements.
- 4) Suppose we have 8 GB of RAM, JVM will allocate 8 GB to it.
- 5) Now, all requirements of Java Appⁿ, are handled by JVM.
- 6) Due to which the allocated memory is called as - 'managed memory'.
- 7) That's why JVM called - 'mini OS'.
- 8) Due to concept of JVM, Java programs are - Platform Independent & Architecture Neutral.
- 9) Java appⁿ is not dependent on underlying OS.

4) Explain Tool-chain of Java





- 1) **JDK** - Java Development Kit
 - Contains no. of tools & resources for developing a Java Appⁿ including:
 - JVM, Compiler, Debugger, Achiever, Applet viewer, Java, java.doc.
- 2) **JRE** - Java Runtime Environment
 - Provides runtime environment for java program.
- 3) **JVM** - Java Virtual Machine
 - Heart of JRE.
 - Responsible for verifying, the bytecode & executing bytecode, line by line ["Interpreter"]
 - Refer Q.3 [for it's working].

4) Why is Java considered as pure OOP?

- 1) Everything in Java is Based on objects & classes.
- 2) Supports all 4 fundamental OOP concepts:
- 1) Encapsulation
 - 2) Abstraction
 - 3) Inheritance
 - 4) Polymorphism
- 3) Does not support - Global functions (must be part of a class)
- 4) No Multiple Inheritance [Uses - Interfaces]
- 5) Memory Management [Garbage Collection]
- [Does not support pointers]

7] What is meant by 'Write Once Run Anywhere' Strategy of Java? [WORA]

-
- Code can be written once & executed on any platform w/o modification.
 - Possible due to - JVM, acts as an abstract layer betⁿ code & underlying OS.
 - Working:

1) Java source code compilation:

Java code (.java file) is compiled by Java Compiler (javac) into an intermediate form called Bytecode (.class file)

2) Platform Independence:

- Does not produce machine-specific code
- Bytecode runs on any JVM
- JVM converts Bytecode into machine code specific to the platform at runtime.

8]	<u>Compiler (javac)</u>	<u>Interpreter (JVM)</u>
Def ⁿ	Translates entire java source code (.java) into bytecode (.class) at once	Executes the bytecode line-by-line
speed	Faster - It translates everything at once	Slower - translates & executes 1 instruction at a time
Execution	Produces bytecode that can run on any JVM	Converts bytecode into machine code specific to the OS
Error Handling	Detects all syntax & compilation errors before execution	Detects & reports errors one by one at runtime
Example	javac Demo.java (Compiles Java source code)	java Demo (Runs the compiled bytecode using the JVM)
Portability	Bytecode is platform independent	JVM interprets bytecode for specific platforms

Native Language

Virtual machine

Based Language

Compiler Language that compiles directly into machine code that runs on a specific OS / hardware.
Code runs directly on the hardware w/o any intermediate layer.

Performance Faster Execution since it runs directly on CPU.

Portability Not portable, needs recompilation for diff platform.

Example C, C++, Assembly language

Compiler Traditional Compilers
e.g. GCC for C, C++

Memory Management Manual memory management
(e.g. using malloc / free in C)

Compiler into an intermediate code that runs on a VM making it platform independent.

Code runs on VM, which converts intermediate code into machine code at runtime.

Slightly slower due to the overhead of VM interpreting the code.

Highly portable, the same code can run on diff platform w/o modification.

Java (JVM), C# (CLR - Common Language Runtime), Python (Python VM)

1st - Compiles into intermediate code (e.g. Bytecode - Java)
2nd - Interprets it

Automatic MM:
(e.g. Garbage Collector - Java)

10] How Java is considered as Secure language?

→ 1] Bytecode Verification : By JVM before execution.

2] No Direct memory Access : (Automatic MM)

- No use of pointers
- Garbage Collection

3] Sandboxing in JVM :

- Runs in restricted environment (sandbox), limits access to system resources

4] Security Manager & Access Control

5] ClassLoader Mechanism :

- Used to separate diff classes

6] Cryptographic Security :

- Built in cryptographic APIs
- Includes - Encryption, decryption, hashing algorithms, digital signatures

7] Exception Handling for secure Code Execution :

8] Protection Against Reverse Engineering

- use obfuscation tools like - ProGuard

9] Secure Communication (SSL/TLS)

- Provides libraries like Java Secure Socket Extensions for secure network connection

10] Platform Independence & Security Patches